

AI-driven methodology for refining and clustering Agile requirements

Samia Nasiri
Moulay Ismail University
Meknes, Morocco
s.nasiri@umi.ac.ma

Mohammed Lahmer
Moulay Ismail University
Meknes, Morocco
m.lahmer@umi.ac.ma

Abstract—Refining the backlog is crucial in the Agile development process for prioritizing work, detecting and resolving problems quickly, and aligning development efforts with project objectives. With automated tools for refining and structuring the backlog, teams can operate more efficiently with a clear understanding of product requirements. In this paper, we present our automated approach for conducting requirements analysis within the Agile Methodology. This method includes the refinement of backlogs through the identification of duplicate user stories and their subsequent clustering. Our method is based on machine learning and natural language processing techniques. To implement our approach, we developed a web-based tool that selects the user stories file, groups them by actor, and applies the unsupervised k-means algorithm to create clusters. We then used word embedding with sentence transformers of BERT to evaluate the similarity rate between each pair of user stories in a cluster. The tool highlights the most similar user stories and facilitates the decision to delete or keep them. The proposed approach was evaluated on a set of case studies using different performance measures. The results demonstrated the effectiveness of our method in detecting duplicate user stories in the backlog.

Keywords— *Artificial intelligence, NLP, Agile methodology*

I. INTRODUCTION

The requirements analysis stage in the System Development Life Cycle is crucial for the success of the process[1]. This occurs because developers must understand the requirements before proceeding to the implementation stage. In this context, User stories are increasingly used to communicate requirements in Scrum. They are semi-structured natural language expressions of requirements at a high level. The text template for user stories has been proposed in many forms. In practice, most practitioners use the following form: "As..., I want..., so that..." [2]. As the number of stakeholders involved in the development process increases, so does the size of the systems developed, resulting in a growing number of user stories. Consequently, the system must be decomposed into a set of sub-systems covering different sets of semantically similar user stories.

There are several approaches and methods for analyzing requirements to generate diagrams such as UML diagrams or goal models. These diagrams are certainly very useful for communication between design teams and for reducing lead times and costs. It is crucial to emphasize that the process of refining requirements constitutes a pivotal factor in diminishing lead times and refining diagrams which can be full of redundancies. It's more advantageous to start refining requirements from the outset, rather than creating diagrams and then considering refining them at a later stage. Requirements. In addition, some authors have studied natural language requirements clustering to decompose the target system at an initial level [3], [4]. The current requirements

clustering approach lacks precision and does not achieve a high level of automation. In contrast, we propose a machine learning-based approach that addresses these challenges. However, clustering of requirements has rarely been considered as a primer for automatically deriving models.

In this paper, We propose a machine learning-based approach for automatically dividing a system into subsystems based on user stories in Scrum.

The organization of requirements within the backlog commenced by categorizing user stories based on the actor's name. This was achieved by applying a set of natural language processing heuristics to extract the actor information from each user story. Once the system is initially decomposed by the actor, we perform a second decomposition for each resulting cluster. Our second clustering of the system relies on the k-means algorithm, which groups semantically similar requirements. We use the BERT model (Bidirectional Encoder Representations from Transformers) to determine any redundancies between the user stories in each cluster. This allows Product Owners and Scrum Masters to more clearly decide on the elimination of redundant user stories from the Product Backlog.

This paper is organized as follows: Section 2 reviews related work. In the next section, we provide a detailed description of our method. In section 4, we describe and analyze the similarity detection between user stories; it also evaluates the performance of our approach. Section 5 presents the discussion of the work. Finally, we end the paper with a conclusion.

II. RELATED WORK

In this section, we present a review of the literature relating to our approach. In [5], the authors proposed an approach that uses semantic similarity measures to suggest possible instances of duplication between user stories. The approach selects the most relevant measure to determine the level of similarity between user stories. The authors' research analyzed semantic similarity measures based on the WordNet WuP lexical database, a similarity measure based on VSM Lin [6], and a measure based on the frequency of common terms Lesk-A [7]. In [8], the authors examined user stories to determine potential information gaps and prevent ambiguities, employing comparisons with prior user stories to detect missing queries. They provided suggestions for users to obtain a better description. The natural language processing is handled by an NLP tool called LingPipe Toolkit. Semantic role labeling was performed to assign roles and actions. In [9], the approach was used to extract information relevant to user stories from recorded conversations between customers and developers. In [3], the work of the authors consisted of clustering specification requirements using first the Vector Space Model (VSM) to

calculate the similarity of functional requirements and then the Agglomerative Hierarchical Clustering (AHC) algorithm to build clusters. In [10], the authors presented a tool-assisted approach for identifying terminological ambiguities between viewpoints, and missing requirements. To this end, their approach involved combining natural language processing with information visualization techniques to help interpret the type of defect. They identified ambiguities and gaps in a set of requirements. The authors used word2vec to detect similarities between terms in the requirements. The visualization displays the requirements graphically by marking the terms used and arranging them in a 2D space according to the viewpoint to which the terms belong. In [11], the authors delivered Sentence-BERT (SBERT). Using a pre-trained modification of the BERT network, SBERT takes benefit of Siamese and triplet network structures by deriving semantically meaningful sentence embeddings that can be evaluated using cosine similarity. In [12], the authors applied Word2vec to calculate word-level similarity, then gathered the requirements into clusters using the Agglomerative Hierarchical Clustering (AHC) algorithm. Word2vec is used for each word following tokenization to eliminate empty words and the radical. The Gensim API was used to extract keywords from each group. This is based on ranking the sentences in the text using a variant of the TextRank algorithm. In [4], the authors utilized a K-means clustering algorithm applied to user stories. In [13], the authors contributed a tool for retrieving time spent in historical and similar user stories. This time helps developers to estimate how much time similar user stories will spend on new projects. To achieve this, the authors employed the NLP algorithm and a pre-trained model built by Google called USE. The model did not differentiate between the opposing “add” and “remove” operations. In [14], the method starts by capturing iStar nodes from semi-structured user stories. Following this, the nodes were merged based on their similarity. Finally, edges between nodes were determined via defined rules. The approach is based on the refinement relationship between iStar nodes. The authors employed a BERT (Bidirectional Encoder Representations from Transformers) model to measure similarity. In [15], the approach was used to detect defects in user stories using 11 quality criteria. Their method combines typical NLP techniques with a new analysis based on iStar modeling. To achieve this, the authors applied NLP techniques such as segmentation, tokenization, POS, as well as the iStar model before verifying the quality of the requirements.

Many approaches for requirements analysis and clustering have been developed so far, falling into two main categories: (1) approaches based on clustering, and (2) approaches focused on identifying duplicates or ambiguities in user stories. Our approach combines these two methods.

III. APPROACH

This section outlines our proposed approach to backlog refinement, which structures the backlog into groups and identifies similar user stories. First, we provide an overview of our proposal. Next, we give a detailed explanation of each step.

Our approach begins by clustering user stories by actor, then opting for an unsupervised clustering approach to be applied within each group. We therefore used the k-means algorithm to generate clusters. In the following step, we

applied the BERT Transformers for each cluster to calculate similarity measures between user stories.

To visualize the clustering and similarity features, we developed an online tool. We used Python and the Flask framework to implement the tool. Fig. 1 shows the steps of the proposed method.

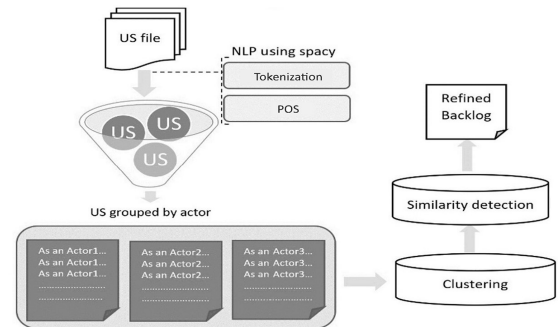


Fig. 1. Steps of our proposed approach.

A. Grouping of user stories by actor

A user story is a definition of a requirement at a very high level, including just enough information; it is the most efficient manner of describing a requirement [2]. In agile project teams, Scrum, XP, Kanban, and other methods are used.

A user story frequently uses the following type of format: As [actor], I want / I can / I can [a goal], so that [a reason] [2].

Before grouping user stories, it was necessary to group requirements by actor. To do this, we extracted the actors using heuristic rules. The rules defined are based on tokenization and POS. We used Python language, and Spacy as an NLP tool.

As the extraction of a compound role comprising three words was not possible with the typed dependency in our previous work [16], we turned to another method which consists of extracting the words following “As a/an”: The role indicator is either an adjective or a noun, except pronouns. Using that solution, we extracted actor nouns independently of their noun type. We then used the rules we had defined to extract actors from user stories, which were stored in a text file. Subsequently, it grouped the requirements by actor and saved each group as a separate file.

Given these user stories:

- As an administrator, I want to search for a specific student, so that I can find the student records.
- As a student, I want to look up my student records, so that I can get a look at the details.

The two user stories are similar in both of them aim to provide access to student records. Their differences lie mainly in the actor implicated: the administrator in the first role, and the student in the second.

Contrary to the methodology described in [13], which concentrated on identifying similarities between user stories without taking into account the “action executor” - i.e. the actor implicated in the action - our approach focused on the

specific roles or actors associated with the actions described in the stories.

Our method classifies user stories according to the actors involved, making it easier to preserve similar stories describing similar actions performed by different roles.

B. Clustering

To implement our method, we used Sentence Transformers to embed sentences. Then, we employed the K-means algorithm to cluster the user stories based on their significance. We labeled the generated clusters using the Gensim library, which also provided us with keywords that describe the domain knowledge embedded in each cluster.

The Gensim library has an implementation of TF-IDF as part of its models, specifically the TF-IDF model module. This module converts documents into a matrix of token counts and calculates TF-IDF weights for each word. These weights can then be used as features for tasks such as text classification, clustering, or similarity calculation.

To refine the backlog, we needed to eliminate duplicate user stories and display only relevant stories with their level of similarity. K-means clustering was used to achieve this by initially setting the value of k and examining the graph to identify the cut-off point where the slope changes. However, we opted for the silhouette calculation method to automate the k -value calculation at the outset. This approach enabled us to find the optimal k -value right from the start, making it easier to group user stories more efficiently. Then, we created a Python code to accomplish this task.

C. User story Similarities and Opposite Meanings Detection

a) Identification of Similar User Story Pairs

To refine the backlog and detect duplicate user stories, we categorized the user stories by actor and then measured the similarity rate between each pair of user stories within each cluster. To achieve this goal, we used a sentence transformer (SBERT) model to measure semantic similarity between the pairs of user stories. Our approach highlights the most similar user stories with rates above 75%.

The SBERT network uses siamese and triplet networks to obtain semantically meaningful sentence embeddings derived from BERT [11]. Similarity measures like cosine similarity or Manhattan/Euclidean distance can be used to determine semantic similarity.

SBERT pre-trained models encode sentences to perform semantic research by calculating the distance between them. Each model is specialized for tasks such as question answering, translation or sentence similarity. They are optimized for various uses and trained on a vast dataset of over a billion pairs [11]. These "sentence transformers" use pre-trained neural networks to convert text into dense semantic vectors. Their strength lies in their ability to capture the full semantic meaning of sentences, beyond individual words. Unlike word-based models, they capture the contextual relationships between words in a sentence, enhancing understanding of overall meaning, which is advantageous for assessing similarity between user stories.

Table I displays models and provides a brief description [11].

TABLE I. EXAMPLE OF SENTENCE TRANSFORMERS WITH DESCRIPTION.

Sentence Transformers	Description	Speed	Performance Sentence Embeddings (14 Datasets)	Model size
all-MiniLM-L6-v2	Sentences and paragraphs are transformed into a dense vector space with 384 dimensions	14200	58.80	80 MB
all-MiniLM-L12-v2		7500	68.70	120 MB
all-mpnet-base-v2	Sentences and paragraphs are transformed into a dense vector space with 768 dimensions	2800	69.57	420 MB
all-distilroberta-v1		4000	59.84	290 MB
paraphrase-multilingual-mpnet-base-v2		2500	53.75	970 MB

We aimed to select the most efficient sentence transformer that provides a higher percentage of similarity among pairs of sentences exhibiting similarities. We focused on comparing user stories that are similar but differ in their operations, ensuring that any dissimilarities were captured as shown in Table II.

We adopted a structured two-phase approach to choose the most suitable model for measuring the similarity between pairs of user stories. First, we carried out a comparative analysis of SBERT models to identify the most efficient model. Then, we extended our evaluation by comparing the SBERT model with other models available in the field.

On the one hand, Table II highlights the application of various SBERT models to detect similarities among pairs of user stories. This table presents the respective similarity rates achieved by applying different SBERT models to some pairs of user stories. On the other hand, Table III illustrates the comparison of the similarity rates obtained from different models, such as Elmo, Word2Vec, and SBERT model, among others. Furthermore, Table III highlights the application of various SBERT models to detect similarities among pairs of user stories. This table presents the respective similarity rates achieved by applying different SBERT models to some pairs of user stories. Table II and Table III illustrate the two-level processes that form an integral part of our analysis approach.

Consider these user stories :

- US1: As a buyer, I am able to remove a product from the list.
- US2: As a buyer, I can drop a product from the list.
- US3: As a buyer, I can edit a product in the list.

Table II shows the similarity score of previous user stories by applying the sentence Transformers.

TABLE II. SIMILARITY SCORES OF USER STORIES US1, US2, AND US3 USING SENTENCE TRANSFORMERS.

Sentence Transformers	Similarity score US1-US2	Similarity score US1-US3
all-MiniLM-L6-v2	0.8180	0.6819656
all-MiniLM-L12-v2	0.8838	0.7911284
all-mpnet-base-v2	0.9025	0.8001925
all-distilroberta-v1	0.9006	0.7651011
paraphrase-mpnet-base-v2	0.9046	0.69480187

TABLE III. SIMILARITY SCORES OF USER STORIES US1, US2, AND US3 USING SIMILARITY MODELS

Models and similarity measures	Similarity score US1-US2	Similarity score US1-US3
USE	0.83511186	0.7759
SBERT(paraphrase-mpnet-base-v2)	0.9046	0.69480187
WORD2VEC	0.84041464	0.7834515
InferSent	0.9690	0.9582
ELMO	0.9342766	0.8595803
Wup similarity	0.66107734	0.3823969

Based on the hierarchical structure of WordNet, Wup similarity evaluates the similarity between words according to their position in the lexical tree. Wup similarity does not take into account the contextual meaning of the words being compared, unlike BERT, ELMO, and InferSent which integrate context to evaluate similarity. This lack of contextual consideration can limit Wup Similarity's ability to capture semantic nuances in a variety of contexts.

Both tables II and III illustrate that the sentence transformer “paraphrase-mpnet-base-v2” exhibits a considerable degree of similarity. We observe operations of US1 and US3, which involve different actions such as “remove” and “edit”. Therefore, the model chosen to evaluate similarity must be able to discern sentences with closely related but divergent meanings. This discernment is essential to avoid any negative impact on our analytical processes and backlog refinement.

To ensure that we had selected the appropriate Sentence Transformer model for our approach, we evaluated by testing several models on two distinct datasets. This evaluation aimed at determining the performance and effectiveness of each model in capturing the semantic meaning of user stories and generating high-quality sentence embeddings. To measure the performance of the models, we used evaluation metrics such as cosine similarity, accuracy, and F1-score. Through this evaluation process, we identified the model that consistently demonstrated superior performance and provided the most meaningful sentence embeddings.

We will provide further details of the evaluation in the next section.

b) Identification of pairs of user stories with opposite meanings

When the similarity threshold is increased, the number of similar user story pairs decreases. To identify similar user stories, a minimum similarity rate of 75% is required while a much higher threshold with similarity rates exceeding 90% is necessary to detect duplicate user stories. In the first range, similar user stories were sometimes found where one story was included in another, providing additional information to the third part of the story. This was mainly due to the poor quality of the user stories. It is only by improving the quality of these stories that the occurrence of such similarities can be reduced.

However, in the second range, although we detected duplicate user stories, they sometimes had opposite meanings that were not captured by sentence similarity models.

Given these user stories :

- US4: As a buyer, I can add a product to the list.
- US5: As a buyer, I can not add a product to the list.
- US6: As a user, I want backend changes for managing enum lists.
- US7: As a user, I want frontend changes for managing enum lists.

Table IV displays the similarity scores of user stories with opposite meanings, which are US4 with US5, and US6 and US7.

TABLE IV. SIMILARITY SCORES OF USER STORIES WITH OPPOSITE MEANINGS

Models and similarity measures	Similarity score US4-US5	Similarity score US6-US7
USE	0.8806	0.9573
SBERT	0.42601973	0.9724
WORD2VEC	0.9803098	0.94255805
InferSent	0.9965	0.9912
ELMO	0.95751846	0.9578713
Wup similarity	0.6326058	0.6258217

In studying the ability of the SBERT model to detect pairs of opposing user stories, it was noted that when two user stories were presenting the same operation, with the same action verb in the format “As [actor], I want [action_verb]” and preceded by a negation “not”, the model was able to identify their opposition. However, if they are not having the same operation, the SBERT model does not detect the dissimilarity. Another case demonstrating opposite meanings is illustrated in the cases described in Table IV. Using sentence transformers, we observed a similarity score of 0.97 between user stories US6 and US7, highlighting substantial similarity despite contrasts in their content. To address this limitation, we developed specific rules. We used Typed dependencies provided by natural language processing (NLP) tools and the Wordnet API to search for synonyms of operations and checked for the presence of negation. Where negation was not explicitly expressed, we opted to detect antonyms for operations, improving the accuracy of detecting pairs of opposing user stories.

As shown in Table IV, we found a similarity of 0.97 between user stories US6 and US7 when using sentence transformers, suggesting high similarity within their differences. Our WordNet approach was able to detect opposing meanings in some pairs of user stories. However, this method showed limited effectiveness for some examined pairs. The rules, we have developed, combine the use of the Wordnet API with typed dependency analysis to identify cases where user stories are initially identified as duplicates, having opposite meanings. The defined rules are based on the Stanford CoreNLP dependencies, such as neg, dobj, nmod, and advcl. The approach comprised the following steps, as presented in the algorithm below.

1. Procedure(story1,story2):
2. Extract_part2(story1,story2)
3. Syn=Extract_synonyms(story1.verb1)

```

4. If [verb2 in Syn and check_negation(story1,story2)]
5. or are_antonyms(verb1, verb2) then
6. If dobj(verb1,obj1) = dobj(verb2,obj2)
7. or nmod(obj1,modifier)= nmod(obj2,modifier) then
8. return "negation found"
9. if story1.Action_advcl= story2.Action_advcl then
10. return "negation found"

```

The algorithm described above is designed to detect negation in a pair of user stories. It worked by first extracting the second part of the user stories that presented the action. Then, it searched for synonyms of the first story's verb (verb1) and checked whether the second story's verb (verb2) is a synonym of the first story's verb (lines 1-3). In line 4, the check_negation(story1, story2) function was implemented by recognizing the negation indicator. In line 5, the algorithm examined whether the verb in story 1 is either a synonym preceded by a negation indicator or a direct antonym of the verb in story 2. If these conditions, along with the direct object or nominal modifier (nmod) had matched between the objects (lines 6-8), the algorithm returns "negation found". Line 9 checked whether the action in the "advcl" dependency is the same in both stories. If this condition, and the previous conditions, are met, the algorithm obtains the result "negation found".

D. Web-Based tool

We developed a web-based tool that allows users to display similar user stories and their similarity percentages in selected clusters. Based on this percentage, users can decide whether to delete similar stories or to keep them. On the homepage, users can select a file containing user stories and click on a button to group them by actor. The file is automatically split into multiple ones, and users can then choose an actor from a drop-down list. Clustering by meaning is then performed, and the similarity rate is displayed. The results are presented in a table format, showing the similarity between each pair of user stories. We set the maximum similarity threshold at 75%.

IV. RESULTS

This section presents the case studies and performance to evaluate the methodology of our approach for detecting more similar user stories.

Course management and Archivespace, which are two distinct datasets, were used to evaluate Sentence transformer models. The first dataset¹, retrieved from the website Mountain Goat Software, consists of 102 user stories. The second dataset², from Archivespace, included 160 user stories. We collected these user stories as a part of the ArchiveSpace software development project.

We carried out evaluations to compare and assess the performance of the "all-MiniLM-L6-v2" and "Paraphrase-mpnet-base-v2" SBERT models, as well as the USE, ELMO, and Word2vec models, using several pairs of user stories.

¹

https://drive.google.com/file/d/1yfeMV12XYU8FgEqwNrteTJIv21W9g_LY/view?usp=share_link

² <https://drive.google.com/file/d/1te9hzKtYC-XILvbwbBOUBJ4emZOALpx/view?usp=sharing>

After observing the similarity scores of different models in distinct ranges, notably between 0.5 and 0.7, as well as those above 0.7, we found that the SBERT model, in particular the "paraphrase-mpnet-base-v2" model, presented more consistent scores when compared to a manual approach. However, to better assess model choice, we used various metrics, including precision, recall, and F1 score [17]. Similarity identification was measured by accuracy, while coverage was measured by recall. We calculated precision, recall, and F1 Score based on true positives, false positives, and false negatives. True positives corresponded to similarities correctly identified by our automated approach, while false positives referred to similarities incorrectly identified. A false negative was a labeled similarity not identified by an automated approach.

The evaluation metrics are measured as follows:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1 Score} = 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$$

Table V shows the precision, recall, and F1 scores for each user story dataset for similarity pairs obtained using SBERT. Table VI displays the similarity detection metric related to USE, Word2vec, and SBERT Models for User Story Pairs of the Archivespace dataset.

TABLE V. EVALUATION METRICS FOR SIMILARITY DETECTION OF USER STORIES OF THE FIRST DATASET

Models	Similarity of user stories pairs					
	TP	FP	FN	Precision	Recall	F1-score
ELMO	7	13	3	35%	70%	46%
Word2vec	14	65	0	17%	100%	29%
USE	5	1	6	83%	45%	58%
SBERT	14	1	0	93%	100%	96%

TABLE VI. COMPARISON OF SIMILARITY DETECTION MODELS FOR USER STORY PAIRS OF ARCHIVE SPACE DATASET

Models	Similarity of user stories pairs					
	TP	FP	FN	Precision	Recall	F1-score
ELMO	4	20	0	16%	100%	27%
Word2vec	4	22	0	15%	100%	26%
USE	1	1	2	50%	33%	39%
SBERT	4	1	0	80%	100%	88%

Table V and Table VI show similar user stories with similarity rates above 90%. When the similarity rate exceeds 90%, the user stories are more similar. The low similarity metrics obtained using Word2Vec to evaluate user stories can be explained by the fact that this model does not capture the semantic complexity of texts as effectively as more recent approaches such as USE and SBERT.

In some cases, one user story was included in another one which added more information in the third part of the user story which was due to poor writing of user stories. If the user stories were of high quality, they could reduce the occurrence of similar user stories. In other cases, the

approach detected two similar user stories with opposite meanings. However, by using Wordnet and our defined rules, we have overcome this shortcoming.

The Universal Sentence Encoder (USE) model [18] is limited in detecting explicit negation. Because of relying on language patterns, it can struggle to capture the opposite or contradictory meaning introduced by negation words like “not”. This can result in inaccurate detection of differences in meaning when negation is present.

Fig. 2 shows a web page that allows the user to download a file containing a set of user stories. Once downloaded, we performed the clustering by actor by clicking the “Clustering by actor” button. The extracted actors were then included in a drop-down list. To perform clustering by meaning, the user needed to click on the “Analyze” button, as illustrated in Fig. 3. It is worth noting that we did not include a text field to specify the number of clusters from the beginning. This is because we automated this process based on silhouette calculations. Fig. 4 illustrates the similarity scores generated for pairs of user stories.

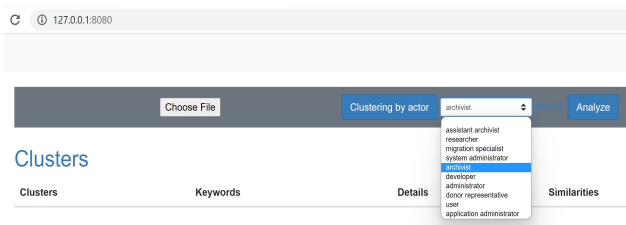


Fig. 2. Items generated in the drop-down list from the user stories file.

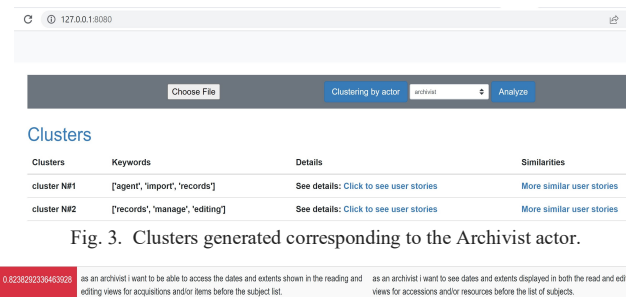


Fig. 3. Clusters generated corresponding to the Archivist actor.

Fig. 4. Some similarity scores generated for pairs of user stories within the Archivist actor's cluster.

Through using AI techniques, particularly SBERT models with our defined rules, our approach offered better detection of duplications and similarities between user stories. Clustering user stories and subsequently labeling these clusters, allowed for an efficient backlog structuring and an improved organization. The refinement process eliminates redundant information and similar functionalities with great precision.

V. DISCUSSION

In this section, we compare our approach with existing state-of-the-art techniques that also use user stories as input data. Although mentioned in [4] and [5], these approaches focus on either clustering or duplicate detection, but do not integrate both, unlike our method. In [5], the WUP similarity is used to determine the level of similarity between user stories. This similarity measure relies on the hierarchical

structure of WordNet and may not capture nuanced semantic meanings as effectively as transformer-based models like SBERT or USE. These models are designed to understand and represent the semantics of complex sentences directly from large-scale text data.

Another study [12] focused on generating use case diagrams from user stories but only considered user stories with a simple structure and did not address the issue of duplicate or similarity detection. In [13], the similarity detection in user stories is ineffective because the method is unable to distinguish between operations such as "delete" and "add" in stories. These operations, which represent basic functionality, remain indistinguishable. The model used to detect similar user stories, namely USE, lacks the ability to discern negations in these stories, such as "not add" and "add" operations.

Our approach incorporated AI techniques, specifically SBERT models and clustering, to refine the backlog. It utilized machine learning to capture semantic similarities as well as the rules we defined to achieve better results.

VI. CONCLUSION

In this paper, we proposed an approach to refine the backlog by detecting similar user stories with a percentage that will help the designer decide to delete or leave the concerned user stories. To achieve this, we used the K-means algorithm to cluster similar user stories. We combined the SBERT model to measure the similarity between user stories with our NLP rules. Our rules uncovered opposing meanings in the identified story pairs, facilitating duplicate detection. Using multiple pairs of user stories, the case studies conducted show that our proposal achieves high performance.

Following this refinement step, designers can derive diagrams or models with reduced redundancies and enhanced clarity.

In future work, we plan to improve our approach by using multiple datasets to improve performance. Furthermore, we aim to define more rules to detect opposite meanings in user stories. Finally, we will focus on detecting non-functional requirements and generating acceptance criteria from them to improve the quality of user stories. It is essential to ensure that user stories remain well-defined and focused on functional aspects, while keeping non-functional requirements, such as performance, security, and usability constraints, specified in the acceptance criteria. By addressing these challenges, we can further enhance the accuracy and efficiency of requirements engineering in software development, ultimately leading to an overall improvement in product quality.

REFERENCES

- [1] Belani H, Vukovic M, Car Z. Requirements engineering challenges in building AI-based complex systems. Proceedings - 2019 IEEE 27th International Requirements Engineering Conference Workshops, REW 2019, Institute of Electrical and Electronics Engineers Inc.; 2019, p. 252–5. <https://doi.org/10.1109/REW.2019.00051>.
- [2] M. Cohn, User Stories Applied: For Agile Software Development. 2004.
- [3] Salman HE, Hammad M, Seriai AD, Al-Shou A. Semantic clustering of functional requirements using agglomerative hierarchical clustering

- Information (Switzerland) 2018;9.
<https://doi.org/10.3390/info9090222>.
- [4] Kumar B, Tiwari UK, Dobhal DC, Negi HS. User Story Clustering using K-Means Algorithm in Agile Requirement Engineering. 2022 International Conference on Computational Intelligence and Sustainable Engineering Solutions (CISES), 2022, p. 1–5. <https://doi.org/10.1109/CISES54857.2022.9844390>.
 - [5] Barbosa R, Silva AEA, Moraes R. Use of Similarity Measure to Suggest the Existence of Duplicate User Stories in the Scrum Process. Proceedings - 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN-W 2016, Institute of Electrical and Electronics Engineers Inc.; 2016, p. 2–5. <https://doi.org/10.1109/DSN-W.2016.27>.
 - [6] Wang J, Dong Y. Measurement of text similarity: a survey. Information. 2020 Aug 31;11(9):421.
 - [7] Banerjee S, Pedersen T. An Adapted Lesk Algorithm for Word Sense Disambiguation Using WordNet. International Conference on Intelligent Text Processing and Computational Linguistics, pp. 136–145, 2002. https://doi.org/10.1007/3-540-45715-1_11.
 - [8] F. S. Bäumer MG. Running out of words: How similar user stories can help to elaborate individual natural language requirement descriptions. Commun. Comput. Inf. Sci., vol. 639, pp. 549–558, Oct. 2016. <https://doi.org/10.1007/978-3-319-46254-7>.
 - [9] Rodeghero P, Jiang S, Armaly A, Mcmillan C. Detecting User Story Information in Developer-Client Conversations to Generate Extractive Summaries. Proceedings - 2017 IEEE/ACM 39th International Conference on Software Engineering, ICSE 2017, 2017, pp. 49–59, <https://doi.org/10.1109/ICSE.2017.13>
 - [10] Dalpiaz F, van der Schalk I, Lucassen G. Pinpointing ambiguity and incompleteness in requirements engineering via information visualization and NLP. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 10753 LNCS, Springer Verlag; 2018, p. 119–35. https://doi.org/10.1007/978-3-319-77243-1_8.
 - [11] Reimers N, Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks 2019.
 - [12] Kochbati T, Li S, Gérard S, Mraidha C. From user stories to models: A machine learning empowered automation. MODELSWARD 2021 - Proceedings of the 9th International Conference on Model-Driven Engineering and Software Development, SciTePress; 2021, p. 28–40. <https://doi.org/10.5220/0010197800280040>.
 - [13] Grzegorz Duszakiewicz A, Glumby Sørensen J, Johansen N, Edison H, Rocha Silva T. On Identifying Similar User Stories to Support Agile Estimation based on Historical Data. 2022.
 - [14] Wu C, Wang C, Li T, Zhai Y. A Node-Merging based Approach for Generating iStar Models from User Stories. Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE, Knowledge Systems Institute Graduate School; 2022, p. 257–62. <https://doi.org/10.18293/SEKE2022-176>.
 - [15] Wang T, Wang C, Li T, Liu Z, Zhai Y. User Story Quality Assessment Based on Multi-dimensional Perspective: A Preliminary Framework. 2022.
 - [16] Nasiri S, Rhazali Y, Lahmer M, Chenfour N. Towards a Generation of Class Diagram from User Stories in Agile Methods. Procedia Comput Sci 2020;170:831–7. <https://doi.org/10.1016/j.procs.2020.03.148>.
 - [17] H. Schütze, C. D. Manning and P. Raghavan, Introduction to Information Retrieval, Cambridge, U.K.:Cambridge University Press, 2008.
 - [18] D. Cer et al., “Universal Sentence Encoder,” Mar. 2018, [Online]. Available: <http://arxiv.org/abs/1803.11175>.